

LoReCS_zsmf

Disclaimer

Bei der Erstellung der folgenden Inhalte wurde viel mit LLMs gearbeitet und dieses Dokument ist stark gekürzt. Keine Gewähr auf Richtigkeit und Vollständigkeit.

Inhalt:

L1

L2

L3

L4

L5

L6

L7

L8

L9

L1

1. Warum Logik?

- **Definition:** Logik ist die Wissenschaft des korrekten Schlussfolgerns ("correct reasoning").
- **Wortherkunft:** Abgeleitet vom griechischen Wort "logos" (Studium, Vernunft).
- **Kernaufgabe:** Aus Prämissen verlässliche Schlussfolgerungen (Conclusions) ziehen.
- **Vorteile:** Bietet Präzision und Vertrauen. Bildet die Basis für viele Anwendungsbereiche.

☰ Anwendungsbereiche der Logik

- **Mathematik:** Nutzt *Axiome* als Startpunkt für Schlussfolgerungen.
- **Legal Reasoning:** z.B. Übersetzung von Gesetzestexten in formale Logik (First-Order Logic - FOL) für "Trustworthy AI".
- **Weitere:** Philosophie, Entscheidungstheorie (wie Menschen/Gruppen Entscheidungen treffen), Biologie/Lebenswissenschaften.

2. Logik in der Informatik (Computer Science)

- Logik spielt in der Informatik eine ähnlich fundamentale Rolle wie die Analysis (Calculus) in der Physik.
- **Zentrale Konzepte:** Syntax, Semantik, Beweistheorie (Proof theory) und automatisches Schließen (Reasoning).
- **Industrierelevanz:** Logik-Fähigkeiten sind in der Industrie extrem gefragt (z.B. zur Minimierung von AI-Halluzinationen bei Amazon Web Services, Neurosymbolische KI).

Beispiel: Software-Korrektheit

```
int* allocateArray(int length)
{
    int i;
    int* array;
    array = malloc(sizeof(int)* length);

    for (i=0; i<=length; i++)
        array[i]=0;
    return array;
}
```

- Anhand eines C-Programms zur Array-Allokation wird gezeigt, dass Fehler oft schwer erkennbar sind (z.B. Schreiben in nicht allokierten Speicher, Null-Pointer-Dereferenzierung).
- **Was bedeutet Korrektheit?** Sie hängt ab von:
 - **Syntax:** Den Regeln und spezifischen Eigenschaften der Programmiersprache.
 - **Semantik:** Der beabsichtigten Bedeutung (intended meaning) des Programms.

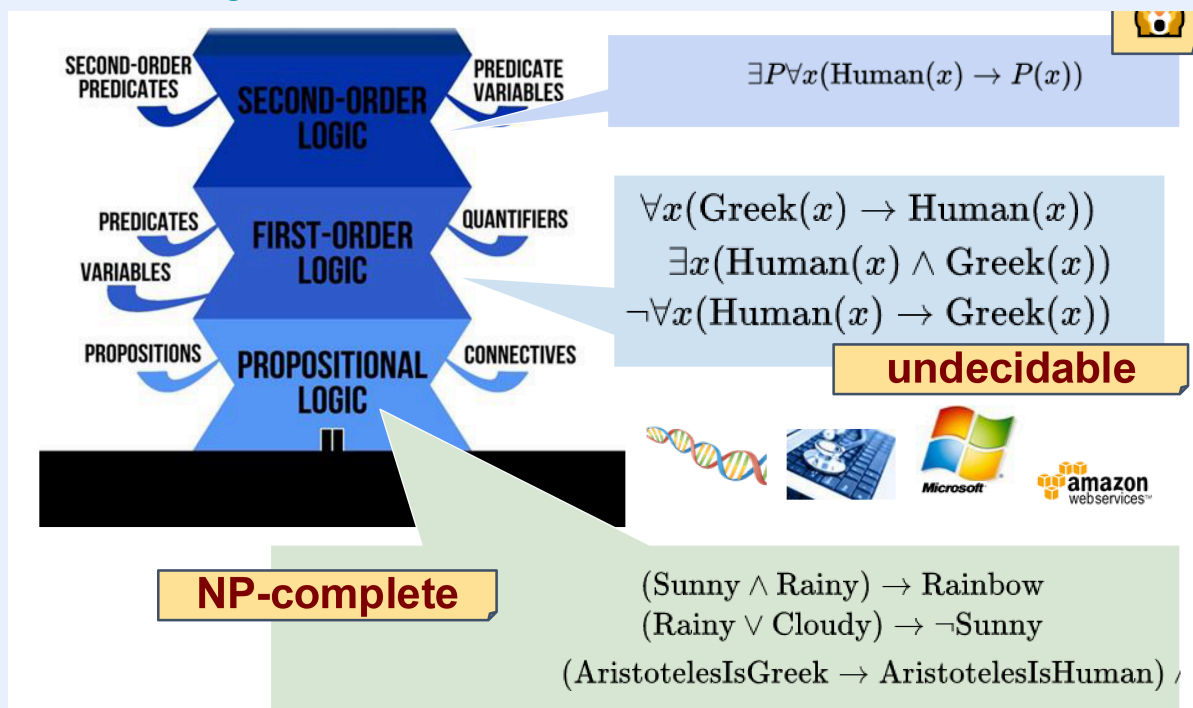
3. Automated Reasoning (KI basierend auf Logik)

- **Grundrezept (seit 1956):** Automatisches Schließen unterstützt menschliches Schließen.
- **1. System modellieren:** Logik als formale Sprache nutzen (Aussagenlogik, Prädikatenlogik 1. Stufe, Prädikatenlogik mit Theorien).
- **2. System trainieren:** Regelbasiertes Software-Reasoning anwenden (z.B. DPLL, Resolution, Tableaux, DPLL(T)).
- **3. Software nutzen:** Effizientes Schließen entlang bekannter Regeln (z.B. SAT Solver wie MiniSat, Theorem Prover wie Vampire, SMT Solver wie Z3).

A Tower of Languages

Übersicht der Logik-Sprachen nach Komplexität und Ausdrucksstärke:

- **Propositional Logic (Aussagenlogik):** Nutzt *Propositions* und *Connectives*. Komplexität: NP-complete. Beispiel: $(Sunny \vee Cloudy) \rightarrow \neg Sunny$
- **First-Order Logic (Prädikatenlogik 1. Stufe):** Führt *Variables*, *Quantifiers* und *Predicates* ein. Komplexität: Undecidable (Unentscheidbar). Beispiel: $\forall x(Greek(x) \rightarrow Human(x))$
- **Second-Order Logic:** Nutzt zusätzlich *Predicate Variables*.



4. Kursablauf & Organisatorisches (LVA 192.033)

Themen des Kurses

- **Propositional Logic:** Syntax, Semantik, Formalisierungen, SAT-Checking (DPLL), Beweiskalküle.
- **First-Order Logic (FOL):** Syntax, Semantik, Beweiskalküle, Unentscheidbarkeit.
- **Satisfiability Modulo Theories (SMT):** SAT-Checking mit Hintergrundtheorien (DPLL(T)), Nelson-Oppen.
- **Plus:** Hands-on Sessions mit entsprechenden Solvern/Provern zu jedem Thema.

Evaluierung & Leistungsbeurteilung

Der Kurs besteht aus drei verpflichtenden Teilen:

1. **Quizzes:** 3 Stück auf TUWEL. Je 20 Min., 15 Fragen (Multiple Choice). Einmal wiederholbar.
Wichtig: Mindestens 25 Punkte insgesamt nötig für die Prüfungszulassung!
2. **Projects (Übungen):** 3 Übungsblätter. Pro Blatt 6-8 (freiwillige) Theorieaufgaben + 1 Tool-basiertes Mini-Projekt. **Wichtig:** Mindestens 2 der 3 Mini-Projekte müssen gelöst und eingereicht werden!
3. **Written Exam (Schriftliche Prüfung):** "Regulated open book" (max. 3 handgeschriebene/gedruckte A4 Blätter erlaubt, keine Elektronik). Termine: 17. Juni 2026 & 28. Sept. 2026. Zum Bestehen sind 50% der Prüfungspunkte nötig.

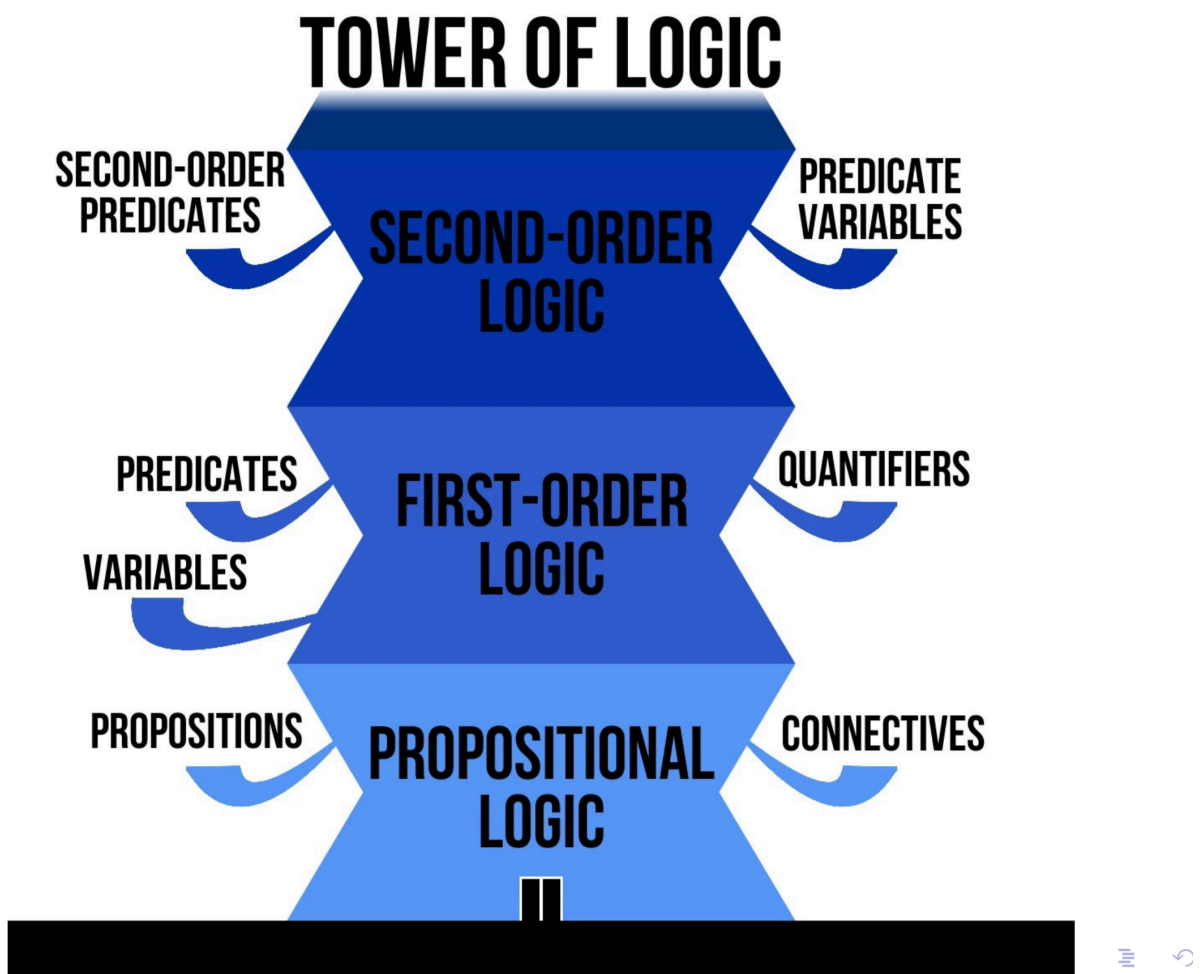
Notenschlüssel & Bonuspunkte

- **Bonus:** 2 Tool-basierte Projekte (je 7.5 Punkte) + Teilnahme an Recap Sessions (1-2 Punkte für In-Class Quizzes).
- **Noten:** Endpunkte = Prüfungspunkte + Bonuspunkte.
 - < 50: Nicht genügend (5)
 - 50 - 62: Genügend (4)
 - 63 - 74: Befriedigend (3)
 - 75 - 86: Gut (2)
 - ≥ 87 : Sehr gut (1)

L2

Einführung in Logik und Schließen

- **Logik** ist das Studium des Schließens (Reasoning).
- Sie bildet die Basis für viele Anwendungen, z.B. in der **Symbolischen KI** (für Wissensrepräsentation, automatisiertes Schließen) und **Subsymbolischen KI** (Integration mit Machine Learning für robustere Modelle).
- **Tower of Logic** (Hierarchie der Logiksysteme): Baut sich auf von Aussagenlogik (Propositions, Connectives) über First-Order Logic (Variables, Quantifiers) bis Second-Order Logic.



Korrektes Schließen (Correct Reasoning)

Ein Schluss (Reasoning) besteht aus einer Reihe von Aussagen, bei denen behauptet wird, dass einige aus den vorherigen folgen.

Ein Schluss ist **korrekt**, wenn die Folgerungen tatsächlich aus den Prämissen abgeleitet werden können.

(Intuition: Logik prüft nur die "Mechanik" der Schlussfolgerung. Selbst wenn die Annahmen völlig absurd sind, kann die logische Ableitung korrekt sein).

Es gibt zwei Hauptansätze, um Korrektheit zu zeigen:

1. **Semantisch:** Es gibt kein Gegenbeispiel (keine Möglichkeit, dass Prämissen wahr, aber Konklusion falsch ist).
 2. **Beweistheoretisch (Proof Theory):** Es gibt einen formalen Beweis von den Prämissen zur Konklusion.
-

Syntax der Aussagenlogik (Propositional Logic)

- **Atomare Aussage:** Ein vollständiger Ausdruck, dem ein Wahrheitswert (0 oder 1) zugeordnet werden kann (z.B. "Der Alarm ging los").
- **Zusammengesetzte Aussage:** Verbindet atomare Aussagen mittels **Boolescher Junktoren** (Und, Oder, Nicht, Wenn...dann).

Wohlgeformte Formeln (WFF)

$$P := At | (P \vee P) | (P \wedge P) | (P \rightarrow P) | (\neg P)$$

- $At = \{A, B, \dots\}$ sind die atomaren Aussagen.
- **Priorität der Operatoren** (um Klammern zu sparen): $\neg > \wedge > \vee > \rightarrow$

- **Subformel:** Jede wohlgeformte Formel Q , die in einer Formel P enthalten ist (P selbst eingeschlossen), ist eine Subformel von P .
-

Semantik der Aussagenlogik

- **Interpretation (v):** Eine Zuweisung von Wahrheitswerten zu atomaren Aussagen $v : At \rightarrow \{0, 1\}$.
- Wahrheitstabellen definieren, wie sich Junktoren verhalten ($\neg, \wedge, \vee, \rightarrow$).

Interpretations $v : At \rightarrow \{0, 1\}$ s.t.

$\neg P$

P	$\neg P$
0	1
1	0

$P \wedge Q$

P	Q	$P \wedge Q$
0	0	0
0	1	0
1	0	0
1	1	1

$P \vee Q$

P	Q	$P \vee Q$
0	0	0
0	1	1
1	0	1
1	1	1

$P \rightarrow Q$

P	Q	$P \rightarrow Q$
0	0	1
0	1	1
1	0	0
1	1	1

Navigation icons: back, forward, search, etc.

Wichtige semantische Konzepte

- **Erfüllbar (Satisfiable / Consistent):** Die Formel hat *mindestens ein* Modell (eine Interpretation v , für die $v(P) = 1$ gilt).
- **Unerfüllbar (Unsatisfiable):** Die Formel hat *kein* Modell (immer falsch).
- **Gültig / Tautologie (Valid):** *Jede* Interpretation ist ein Modell für P (wird notiert als $\models P$).

Dualitäts-Theorem

Eine Formel P ist eine Tautologie, genau dann wenn $\neg P$ unerfüllbar ist.

- **Komplexität:** Das Testen auf Erfüllbarkeit (SAT) ist in der Komplexitätsklasse **NP**. Ein Entscheidungsproblem ist co-NP, wenn sein Komplement in NP liegt.

Semantische Konsequenz ($\models$)

- Beschreibt, ob eine Konklusion zwingend aus einer Menge von Prämissen folgt.
- Sei Γ eine Menge von WFFs und Q eine Formel. Q ist eine **semantische Konsequenz** von Γ ($\Gamma \models Q$), wenn es **kein Gegenbeispiel** gibt.

- Formal: Für jede Interpretation v gilt: Wenn alle Prämissen in Γ wahr sind ($v(P_i) = 1$), dann muss auch Q wahr sein ($v(Q) = 1$).

Dualität & Deduktionstheorem

- Dualität: $\Gamma \models Q$ gilt gdw. $\Gamma \cup \{\neg Q\}$ **unerfüllbar** ist. (*Intuition: Es ist unmöglich, dass die Prämissen wahr sind UND die Konklusion falsch ist*).
- Deduktionstheorem: $\Gamma \models Q$ gilt gdw. die Implikation $(\bigwedge_{P_i \in \Gamma} P_i) \rightarrow Q$ **gültig** ist.

Äquivalenz von Formeln ($\equiv$)

- Zwei Formeln P und Q sind **äquivalent** ($P \equiv Q$), wenn sie unter allen Interpretationen denselben Wahrheitswert haben ($v(P) = v(Q)$ für alle v).

Wichtige Äquivalenz-Regeln:

- Idempotenz: $P \wedge P \equiv P$ und $P \vee P \equiv P$
- Kommutativität: $P \vee Q \equiv Q \vee P$
- Assoziativität: $(P \vee Q) \vee R \equiv P \vee (Q \vee R)$
- Absorption: $P \vee (P \wedge Q) \equiv P$ und $P \wedge (P \vee Q) \equiv P$
- Distributivität: $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$
- De Morgan: $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ sowie $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
- Doppelte Negation: $\neg\neg P \equiv P$
- Implikation auflösen: $A \rightarrow B \equiv \neg A \vee B$
- Äquivalenz auflösen: $A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$
- Wahr / Falsch: $\top \equiv \neg A \vee A$ (Tautologie), $\perp \equiv A \wedge \neg A$ (Widerspruch)

Anwendung: Digital Design (Logikgatter)

Logikschaltungen können direkt in Aussagenlogik übersetzt werden (AND, OR, NOT, NAND, XOR, etc.). Äquivalenzregeln helfen dabei, Schaltungen zu vereinfachen.

Equivalence of formulas: an application

The correspondence between digital logic circuits and propositional logic has been known for a long time.

Gate	Symbol	Rule
NOT		$0 \rightarrow 1$ $1 \rightarrow 0$
AND		$00 \rightarrow 0$ $01 \rightarrow 0$ $10 \rightarrow 0$ $11 \rightarrow 1$
OR		$00 \rightarrow 0$ $01 \rightarrow 1$ $10 \rightarrow 1$ $11 \rightarrow 1$
NAND		$00 \rightarrow 1$ $01 \rightarrow 1$ $10 \rightarrow 1$ $11 \rightarrow 0$
NOR		$00 \rightarrow 1$ $01 \rightarrow 0$ $10 \rightarrow 0$ $11 \rightarrow 0$
XOR		$00 \rightarrow 0$ $01 \rightarrow 1$ $10 \rightarrow 1$ $11 \rightarrow 0$
XNOR		$00 \rightarrow 1$ $01 \rightarrow 0$ $10 \rightarrow 0$ $11 \rightarrow 1$

Digital Design	Propositional Logic
circuit	formula
logic gate	propositional connective
input wire	atom
voltage level	truth value

Many problems in circuit designs and verification can be reduced to equivalence of formulas....

Navigation icons: back, forward, search, etc.

Normalformen

- **Literal:** Eine atomare Aussage (A) oder ihre Negation ($\neg A$).

CNF und DNF

- **CNF (Conjunctive Normal Form):** Formel ist ein "Und" von "Oder"s. $P = P_1 \wedge \dots \wedge P_n$, wobei jedes P_i eine Disjunktion ($\vee$) von Literalen ist.
- **DNF (Disjunctive Normal Form):** Formel ist ein "Oder" von "Und"s. $P = P_1 \vee \dots \vee P_n$, wobei jedes P_i eine Konjunktion ($\wedge$) von Literalen ist.

- Zu jeder WFF gibt es eine äquivalente CNF und DNF.

Wie berechnet man eine Normalform?

1. Durch Äquivalenzregeln:

- Alle Junktoren außer $\vee, \wedge, \neg$ eliminieren (z.B. Implikationen umwandeln).
- De Morgan und doppelte Negation anwenden (damit Negationen direkt bei den Variablen stehen).

- Distributivgesetz anwenden, um die Formel umzustrukturieren.

2. Durch Wahrheitstabellen (Speziell für DNF):

- Wahrheitstabelle für P aufstellen.
 - Alle Zeilen betrachten, in denen $v(P) = 1$.
 - Für jede solche Zeile einen "Und"-Block (Konjunktion) bilden: Wenn eine Variable 1 ist, schreibe A_i , wenn sie 0 ist, schreibe $\neg A_i$.
 - All diese Blöcke mit "Oder" ($\vee$) verbinden.
-

Funktionale Vollständigkeit (Functional Completeness)

- Jeder n -stellige Junktorkann als Funktion $f : \{0, 1\}^n \rightarrow \{0, 1\}$ gesehen werden.
- Eine Menge von Junktoren ist **funktional vollständig**, wenn sich *jeder erdenkliche* Junktorkann nur durch die Kombination dieser Junktoren darstellen lässt.
- Das Set $\{\wedge, \vee, \neg\}$ ist funktional vollständig.
- Sogar die kleineren Sets $\{\wedge, \neg\}$ und $\{\vee, \neg\}$ reichen aus (da man z.B. $\vee$ durch De Morgan aus $\wedge$ und $\neg$ bauen kann).

L3

Aussagenlogik (Propositional Logic) - Syntax & Semantik

- **Zweck:** Formalisiert Aussagen, die entweder wahr (1) oder falsch (0) sein können.
- Gegeben ist eine abzählbare Menge an **atomaren Propositionen** (Atome / Boolesche Variablen), meist notiert als Kleinbuchstaben $p, q, r, \dots$

Wohlgeformte Formeln (WFF - Well-Founded Propositional Formulas)

- Jede boolesche Variable an sich ist eine Formel.
- $\top$ (Verum/Wahr) und $\perp$ (Falsum/Falsch) sind Formeln.
- Wenn $P_1, \dots, P_n$ Formeln sind, dann sind auch $(P_1 \wedge \dots \wedge P_n)$ und $(P_1 \vee \dots \vee P_n)$ Formeln.
- Wenn P eine Formel ist, ist $(\neg P)$ eine Formel.
- Wenn P_1 und P_2 Formeln sind, sind $(P_1 \rightarrow P_2)$ und $(P_1 \leftrightarrow P_2)$ Formeln.

Junktoren (Connectives) & Präzedenz

- Um übermäßige Klammern in Formeln zu vermeiden, gibt es eine **feste Rangfolge der Operatoren** (höchste Präzedenz bindet am stärksten).
- --hier

Connectives and Their Precedence

We want to avoid expressions cluttered with parentheses.
Therefore, we assign precedence to operators and **use the precedence to disambiguate expressions.**

Connective	Name	Precedence
$\top$	verum	
$\perp$	falsum	
$\neg$	negation	5
$\wedge$	conjunction	4
$\vee$	disjunction	3
$\rightarrow$	implication	2
$\leftrightarrow$	equivalence	1

- *Reihenfolge*: $\top, \perp (5) > \neg (4) > \wedge (3) > \vee (2) > \rightarrow (1) > \leftrightarrow (1)$.

Parsing von Formeln

Man kann Formeln *Inside-out* (von der höchsten zur niedrigsten Präzedenz) oder *Outside-in* (von der niedrigsten zur höchsten) auflösen. Das Endergebnis ist stets dasselbe, unabhängig von der Reihenfolge.

Interpretationen & Operationstabellen

- Die *Semantik* einer Formel wird definiert, indem den Variablen Werte zugewiesen werden.
- Eine *Interpretation* I für eine Variablenmenge S ist ein Mapping (auch *Truth Assignment* genannt): $I : S \rightarrow \{1, 0\}$.
- Der Wahrheitswert einer komplexen Formel wird eindeutig durch die Wahrheitswerte ihrer Teilformeln bestimmt (*Kompositionalität*).
- --hier

Interpreting Formulas – Operation Tables

$\wedge$	1	0	$\vee$	1	0	$\neg$	
1	1	0	1	1	1	1	0
0	0	0	0	1	0	0	1
	$\rightarrow$	1	0		$\leftrightarrow$	1	0
	1	1	0		1	1	0
	0	1	1		0	0	1

Every connective is a **function** on truth values.

The truth value of a formula is **uniquely determined** by the truth values of its subformulas.

Zentrale semantische Eigenschaften

Sei P eine Formel und I eine Interpretation.

- **Erfüllbarkeit (Satisfiability):** P ist erfüllbar, wenn es *mindestens ein* Modell gibt (d.h. es gibt ein I mit $I \models P$, also $I(P) = 1$). *(Ergänzung: Ein Modell macht die Formel wahr).*
- **Gültigkeit / Tautologie (Validity):** P ist gültig, wenn *jede* Interpretation ein Modell von P ist (immer wahr).
- **Äquivalenz:** Zwei Formeln P und Q sind äquivalent ($P \equiv Q$), wenn sie exakt dieselben Modelle haben.

Zusammenhänge der Eigenschaften

- P ist gültig $\Leftrightarrow \neg P$ ist unerfüllbar.
- P ist erfüllbar $\Leftrightarrow \neg P$ ist nicht gültig.
- P ist gültig $\Leftrightarrow P \equiv \top$.
- $P \equiv Q \Leftrightarrow (P \leftrightarrow Q)$ ist gültig.
- $P \equiv Q \Leftrightarrow \neg(P \leftrightarrow Q)$ ist unerfüllbar.

Formelauswertung (Formula Evaluation)

Equivalent Replacement (Äquivalentes Ersetzen)

Wenn $B_1 \equiv B_2$, dann gilt auch $A[B_1] \equiv A[B_2]$.

(Ergänzung: Ersetzt man in einer Formel A eine Teilformel B_1 durch eine logisch äquivalente Formel B_2 , bleibt die resultierende Gesamtformel äquivalent.)

Syntaktischer Auswertungs-Algorithmus

Man kann eine Formel rein syntaktisch auswerten, ohne Wahrheitstabellen zeichnen zu müssen:

1. Ersetze jedes Vorkommen einer Variablen p in der Formel durch $\top$ (wenn $I \models p$) oder $\perp$ (wenn $I \not\models p$).
2. Vereinfache die Formel anschließend mithilfe eines Systems aus **Rewrite Rules** (Umschreibungsregeln) so lange, bis nur noch $\top$ oder $\perp$ übrig ist.

- --hier

Rewrite Rules for Evaluating a Formula

Suppose that we have a formula consisting only of $\perp$ and $\top$. One can note that every formula of this form different from $\perp$ and $\top$ can be “simplified” to a smaller equivalent formula.

For example, every formula of the form $A \rightarrow \top$ is equivalent to a simpler formula $\top$.

This simplification process can be formalised as a **rewrite rule system**:

$\top \wedge \dots \wedge \top \Rightarrow \top$ $\perp \wedge A_1 \wedge \dots \wedge A_n \Rightarrow \perp$

$\top \vee A_1 \vee \dots \vee A_n \Rightarrow \top$ $\perp \vee \dots \vee \perp \Rightarrow \perp$
--

$\neg \top \Rightarrow \perp$ $\neg \perp \Rightarrow \top$

$A \rightarrow \top \Rightarrow \top$ $\perp \rightarrow A \Rightarrow \top$ $\top \rightarrow \perp \Rightarrow \perp$

$\top \leftrightarrow \top \Rightarrow \top$ $\top \leftrightarrow \perp \Rightarrow \perp$ $\perp \leftrightarrow \top \Rightarrow \perp$ $\perp \leftrightarrow \perp \Rightarrow \top$

[einfügen-- \(Übersicht der Rewrite Rules\)](#)

- **Beispiele für Regeln:** $A \wedge \perp \Rightarrow \perp$, $A \rightarrow \top \Rightarrow \top$, $\neg \perp \Rightarrow \top$.

Das SAT-Problem (Propositional Satisfiability)

- **Problemstellung:** Gegeben eine Formel A , prüfe, ob sie erfüllbar ist. Falls ja, finde ein *Satisfying Assignment* (ein Modell).
- Es ist eines der bekanntesten kombinatorischen Probleme der Informatik und das erste Problem, für das **NP-Vollständigkeit** (NP-complete) bewiesen wurde.

☰ SAT-Formalisierung: Logik-Olympiade

- **Die Aufgabe:** 3 Finalisten (Louis, Rene, Johannes). Wer belegte welchen Platz?
- **Die Aussagen:** Isaac sagt: "Louis 1., Rene 2." ($L_1 \wedge R_2$). Albert sagt: "Johannes 1., Louis 2." ($J_1 \wedge L_2$).
- **Die Bedingung:** Beide haben je eine wahre und eine falsche Aussage gemacht.
- **Formalisierung in Klauseln (Clauses):** Isaac $\rightarrow (\neg L_1 \vee \neg R_2) \wedge (L_1 \vee R_2)$. Albert $\rightarrow (\neg J_1 \vee \neg L_2) \wedge (J_1 \vee L_2)$.
- Zusätzlich braucht es **Constraints** (jeder Platz nur 1x vergeben, jede Person nur 1 Platz), was in weiteren Klauseln resultiert.

SAT Formalisation - Logic Olympiad Puzzle

Isaac reported that **Louis** won the olympiad, while **Rene** was second.

$$L_1 \wedge R_2$$

Albert reported that **Johannes** won the olympiad, while **Louis** was second.

$$J_1 \wedge L_2$$

Neither Isaac nor Albert had given a correct report of the results of the olympiad. Each of them had given one **true** and one **false** statement.

from Isaac: $\neg L_1 \vee \neg R_2$
 $L_1 \vee R_2$

from Albert: $\neg J_1 \vee \neg L_2$
 $J_1 \vee L_2$

Hidden: Exactly one of the three contestants was first (respectively, second and third). This gives us the following 12 clauses:

$$L_1 \vee R_1 \vee J_1 \quad \neg L_1 \vee \neg R_1 \quad \neg L_1 \vee \neg J_1 \quad \neg R_1 \vee \neg J_1$$

$$L_2 \vee R_2 \vee J_2 \quad \neg L_2 \vee \neg R_2 \quad \neg L_2 \vee \neg J_2 \quad \neg R_2 \vee \neg J_2$$

$$L_3 \vee R_3 \vee J_3 \quad \neg L_3 \vee \neg R_3 \quad \neg L_3 \vee \neg J_3 \quad \neg R_3 \vee \neg J_3$$

Similarly, each of the contestants cannot take more than one place.

$$L_1 \vee L_2 \vee L_3 \quad \neg L_1 \vee \neg L_2 \quad \neg L_1 \vee \neg L_3 \quad \neg L_2 \vee \neg L_3$$

$$R_1 \vee R_2 \vee R_3 \quad \neg R_1 \vee \neg R_2 \quad \neg R_1 \vee \neg R_3 \quad \neg R_2 \vee \neg R_3$$

$$J_1 \vee J_2 \vee J_3 \quad \neg J_1 \vee \neg J_2 \quad \neg J_1 \vee \neg J_3 \quad \neg J_2 \vee \neg J_3$$

The resulting set of formulas is **satisfiable** and has only one model:

That is, **Johannes** won, **Rene** was second, and **Louis** came third.

How to check satisfiability?

Wahrheitstabellen (Truth Tables)

- Um die Erfüllbarkeit festzustellen, wertet man Formeln iterativ über Interpretationen aus.
- Problem:** Eine Formel mit n Variablen hat 2^n verschiedene Interpretationen, die Tabelle wächst exponentiell.

Kompakte Wahrheitstabellen (Compact Truth Tables)

- Idee:** Oft kann eine Formel ausgewertet werden, indem man nur die Werte einer **Teilmenge** der Variablen betrachtet (z.B. wenn eine Seite einer AND-Verknüpfung bereits 0 ist, ist der Rest egal).

Compact Truth Table

Idea: we can sometimes evaluate a formula based on values of only a **subset of all variables**.

subformula	l_2	l_3	l_4	l_1
$\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$	0	0	0	0
$(p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r)$	1	1	1	1
$p \rightarrow r$	1	0	0	1
$(p \rightarrow q) \wedge (p \wedge q \rightarrow r)$		0	0	
$p \wedge q \rightarrow r$		1	0	1
$p \rightarrow q$		0	1	
$p \wedge q$		0	1	
p	0	1	1	
q		0	1	
r	0	0	0	1

The formula is unsatisfiable.

Note: the size of the compact table (but not the result) depends on the order of atoms!

The ideas of **guessing variable values** (or **case analysis**) and **propagation** are the key ideas in nearly all propositional satisfiability algorithms.

•
einfügen--**

- Die Größe der kompakten Tabelle hängt stark von der Reihenfolge der Variablen ab (das Endergebnis ändert sich dadurch aber nicht).
- Die Konzepte des "Werte ratens" (Guessing / Case Analysis) und der Propagierung sind die Schlüsselkonzepte in fast allen modernen SAT-Algorithmen.

L4

Logic and Reasoning - Splitting

 Überblick

Diese Mitschrift behandelt fortgeschrittene Entscheidungsverfahren für die Erfüllbarkeit in der Aussagenlogik (Propositional Satisfiability), mit besonderem Fokus auf **Splitting** und **Polaritäten**.

Compact Truth Tables (Kompakte Wahrheitstabellen)

- **Idee:** Manchmal reicht es aus, eine Formel nur basierend auf den Werten einer Teilmenge aller Variablen auszuwerten.
- Die Größe der kompakten Tabelle hängt von der **Reihenfolge der Atome** ab (das finale Ergebnis jedoch nicht).
- Das Raten von Variablenwerten (Fallunterscheidung / Case Analysis) und das anschließende Propagieren sind die Schlüsselideen in fast allen Algorithmen zur Erfüllbarkeit in der Aussagenlogik.

Compact Truth Table

Idea: we can sometimes evaluate a formula based on values of only a **subset of all variables**.

subformula					l_2	l_3	l_4	l_1
$\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$					0	0	0	0
$(p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r)$					1	1	1	1
$p \rightarrow r$					1	0	0	1
$(p \rightarrow q) \wedge (p \wedge q \rightarrow r)$						0	0	
$p \wedge q \rightarrow r$						1	0	1
$p \rightarrow q$						0	1	
$p \wedge q$						0	1	
p	p	p	p	p	0	1	1	
	q	q				0	1	
			r	r	0	0	0	1

The formula is **unsatisfiable**.

Note: the size of the compact table (but not the result) depends on the order of atoms!

The ideas of **guessing variable values** (or **case analysis**) and **propagation** are the key ideas in nearly all propositional satisfiability algorithms.

Entscheidungsverfahren (Decision Procedures)

Neben Wahrheitstabellen gibt es komplexere Verfahren:

- Splitting
- DPLL
- Resolution
- Semantic Tableaux
- BDDs (Binary Decision Diagrams)

Splitting: Die Grundidee

Beim Splitting wählt man eine Variable p und führt eine Fallunterscheidung (Case Analysis) durch:

- Wenn p falsch ist, ersetze p durch $\perp$.
- Wenn p wahr ist, ersetze p durch $\top$.
- $A_p^\perp$ und $A_p^\top$ bezeichnen die Formeln, die man erhält, wenn man in der Formel A alle Vorkommen von p durch $\perp$ bzw. $\top$ ersetzt.

Splitting über Atome (Lemma & Theorem)

Sei p ein Atom, A eine Formel und I eine Interpretation.

- Wenn $I \models p$, dann ist A in I äquivalent zu $A_p^\top$.
- Wenn $I \not\models p$, dann ist A in I äquivalent zu $A_p^\perp$.
- **Theorem:** Eine Formel A ist genau dann erfüllbar (satisfiable), wenn mindestens eine der beiden Formeln $A_p^\top$ oder $A_p^\perp$ erfüllbar ist.

Simplification Rules (Vereinfachungsregeln)

Sobald eine Formel $\top$ oder $\perp$ enthält, muss sie vereinfacht werden. Die Regeln greifen, bis nur noch $\perp$, $\top$ oder eine Formel ohne diese Konstanten übrig ist.

Regeln für $\top$:

- $\neg \top \Rightarrow \perp$
- $\top \wedge A_1 \wedge \dots \wedge A_n \Rightarrow A_1 \wedge \dots \wedge A_n$
- $\top \vee A_1 \vee \dots \vee A_n \Rightarrow \top$
- $A \rightarrow \top \Rightarrow \top$
- $\top \rightarrow A \Rightarrow A$
- $A \leftrightarrow \top \Rightarrow A$ und $\top \leftrightarrow A \Rightarrow A$

Regeln für $\perp$:

- $\neg \perp \Rightarrow \top$
- $\perp \wedge A_1 \wedge \dots \wedge A_n \Rightarrow \perp$
- $\perp \vee A_1 \vee \dots \vee A_n \Rightarrow A_1 \vee \dots \vee A_n$
- $A \rightarrow \perp \Rightarrow \neg A$ und $\perp \rightarrow A \Rightarrow \top$
- $A \leftrightarrow \perp \Rightarrow \neg A$ und $\perp \leftrightarrow A \Rightarrow \neg A$

Der Splitting Algorithmus & Tree

Algorithmus `split(G)`

1. $G := \text{simplify}(G)$
2. Wenn $G = \top$, return "satisfiable"
3. Wenn $G = \perp$, return "unsatisfiable"
4. Wähle ein Atom und einen Wahrheitswert: $(p, b) := \text{select}(G)$
5. **Rekursion:** Überprüfe basierend auf b (0 oder 1) zuerst einen Fall. Wenn dieser "satisfiable" liefert, ist die Formel erfüllbar. Ansonsten überprüfe den anderen Fall ($G_p^\perp$ bzw. $G_p^\top$).

Splitting Algorithm

```

procedure split( $G$ )
parameters: function select
input: formula  $G$ 
output: "satisfiable" or "unsatisfiable"
begin
   $G := \text{simplify}(G)$ 
  if  $G = \top$  then return "satisfiable"
  if  $G = \perp$  then return "unsatisfiable"
   $(p, b) := \text{select}(G)$ 
  case  $b$  of
    1  $\Rightarrow$ 
      if split( $G_p^\top$ ) = "satisfiable"
        then return "satisfiable"
      else return split( $G_p^\perp$ )
    0  $\Rightarrow$ 
      if split( $G_p^\perp$ ) = "satisfiable"
        then return "satisfiable"
      else return split( $G_p^\top$ )
  end

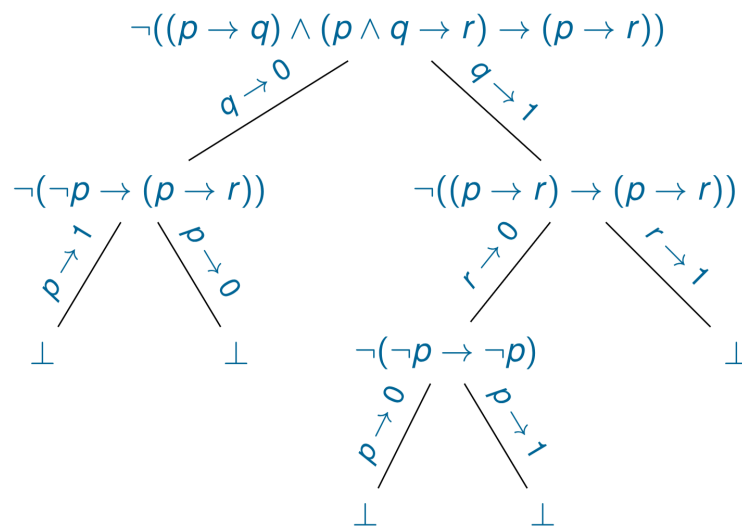
```

Splitting Tree (Baumstruktur):

- **Knoten:** Teilformeln
- **Kanten:** Gelabelt mit den gewählten Zuweisungen der Atome (Splitting Cases)
- **Blätter:** $\perp$ oder $\top$
- (Ergänzung: Ein Ast, der bei $\top$ endet, liefert ein Modell für die Formel. Man sammelt einfach die Zuweisungen auf diesem Pfad).
- Die Größe des Baums hängt stark von der Reihenfolge der gewählten Atome ab.
- Funktioniert auch mit endlichen Mengen von Formeln.

Splitting Algorithm, Example, Splitting Tree

Tree nodes: subformulas; **edges:** labelled by splitting cases over atoms; **leaves:** $\perp$, $\top$



The formula is **unsatisfiable**.

Parse Tree (Ableitungsbaum) & Positionen

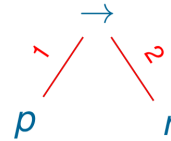
Um Formeln strukturiert zu verarbeiten, nutzt man Parse Trees.

- **Knoten:** Logische Verknüpfungen (Connectives) oder Atome.
- **Kanten:** Verbinden Teilformeln mit ihren Eltern-Knoten. Sie sind oft mit Zahlen (1, 2, ...) gelabelt, die angeben, welches Argument der Verknüpfung es ist (z.B. linke oder rechte Seite).
- **Blätter:** Atome.

Parse Tree

Consider the formula $p \rightarrow r$. Its **parse tree** is:

- Nodes: Connectives or atoms
- **Edges**: Connect subformulas with their parents, **labelled** with integers $1, 2, \dots$ denoting connective arguments
- Leaves: Atoms



Positionen in Teilformeln

Eine Position π ist eine Sequenz aus positiven Ganzzahlen $a_1. a_2. \dots a_n$.

- ϵ (leere Position) repräsentiert die gesamte Formel ($n = 0$).
- Sei $A|_{\pi} = B$ (die Teilformel an Position π):
 - Ist $B = B_1 \wedge \dots \wedge B_n$ oder $B = B_1 \vee \dots \vee B_n$, dann ist $A|_{\pi.i} = B_i$.
 - Ist $B = \neg B_1$, dann ist $A|_{\pi.1} = B_1$.
 - Ist $B = B_1 \rightarrow B_2$, dann ist $A|_{\pi.1} = B_1$ und $A|_{\pi.2} = B_2$.
 - Ist $B = B_1 \leftrightarrow B_2$, dann ist $A|_{\pi.i} = B_i$.

Polarität von Teilformeln (Polarity)

Eine wichtige Optimierung beim Splitting ist die Beachtung der **Polarität**. Sie formalisiert die Beziehung zwischen den Wahrheitswerten einer Formel und ihrer Teilformeln.

Polaritätswerte

Es gibt drei Werte: **1** (positiv/monoton), **-1** (negativ/anti-monoton) und **0** (weder noch).

- Atom mit nur Polarität 1: Hier muss man nicht splitten! Man setzt einfach direkt $p \mapsto 1$.
- Atom mit nur Polarität -1: Hier setzt man einfach direkt $p \mapsto 0$.

Monotonie logischer Verknüpfungen:

Man definiert eine Ordnung der Wahrheitswerte mit $0 < 1$ (daraus folgt: $0 \leq 0$, $0 \leq 1$, $1 \leq 1$, aber *nicht* $1 \leq 0$).

- Monoton (1): $\wedge, \vee, \rightarrow$ (2. Argument / rechte Seite).
- Anti-monoton (-1): $\neg, \rightarrow$ (1. Argument / linke Seite).
- Weder noch (0): $\leftrightarrow$.

Ersetzungen basierend auf Polarität:

- *Equivalent Replacement*: Wenn $I \models B_1 \leftrightarrow B_2$, dann $I \models A[B_1] \leftrightarrow A[B_2]$.
 - Spezialfall: Wenn $I \models p \leftrightarrow \top$, dann $I \models A[p] \leftrightarrow A_p^\top$.
- *Monotonic Replacement*: Wenn $I \models B_1 \rightarrow B_2$ und B_1 hat eine positive Polarität, dann gilt $I \models A[B_1] \rightarrow A[B_2]$.
 - Spezialfall: Wenn $I \models p \rightarrow \top$ und p hat positive Polarität, dann $I \models A[p] \rightarrow A_p^\top$.

L5

Decision Procedures for Propositional Satisfiability

 Fortgeschrittene Entscheidungsverfahren

Neben einfachen Wahrheitstabellen gibt es effizientere Verfahren zur Überprüfung der Erfüllbarkeit (Satisfiability) von Formeln:

- Splitting mit Polaritäts-Optimierungen
- DPLL (Davis-Putnam-Logemann-Loveland)
- Resolution
- Semantic Tableaux

Polarität von Teilformeln (Polarity of Subformulas)

Die Polarität formalisiert die Beziehung zwischen dem Wahrheitswert einer Gesamtformel und dem Wahrheitswert ihrer Teilformeln.

Grundidee & Monotonie:

- Ordnung von Wahrheitswerten: $0 < 1$ (Daraus folgt: $0 \leq 0$, $0 \leq 1$, $1 \leq 1$, aber nicht $1 \leq 0$).
- Verhalten logischer Junktoren bezüglich dieser Ordnung:
 - **Monoton (Polarität 1):** $\wedge, \vee, \rightarrow$ (nur das 2. Argument, also die Konklusion). *(Intuitiv: Wird der Teil wahrer, wird auch das Ganze tendenziell wahrer).*
 - **Anti-monoton (Polarität -1):** $\neg, \rightarrow$ (nur das 1. Argument, also die Prämisse). *(Intuitiv: Wird der Teil wahrer, wird das Ganze tendenziell falscher).*
 - **Weder noch (Polarität 0):** $\leftrightarrow$.

 Polarität an einer Position π

Notation: $pol(A, \pi)$ gibt die Polarität der Teilformel B an Position π in Formel A an.

- **Wurzel:** Formel A an leerer Position ϵ hat Polarität 1.
- **$\wedge$ und $\vee$:** Polarität wird vererbt ($pol(A, \pi.i) = pol(A, \pi)$).
- **$\neg$:** Polarität kehrt sich um ($pol(A, \pi.1) = -pol(A, \pi)$).
- **$\rightarrow$:** Prämisse kehrt Polarität um ($-pol(A, \pi)$), Konklusion behält sie bei ($pol(A, \pi)$).
- **$\leftrightarrow$:** Beide Seiten erhalten Polarität 0.

Ein Vorkommen heißt **positiv**, wenn Polarität = 1, und **negativ**, wenn Polarität = -1.

Der "Coloring Algorithm" (Färbungs-Algorithmus zur Bestimmung der Polarität)

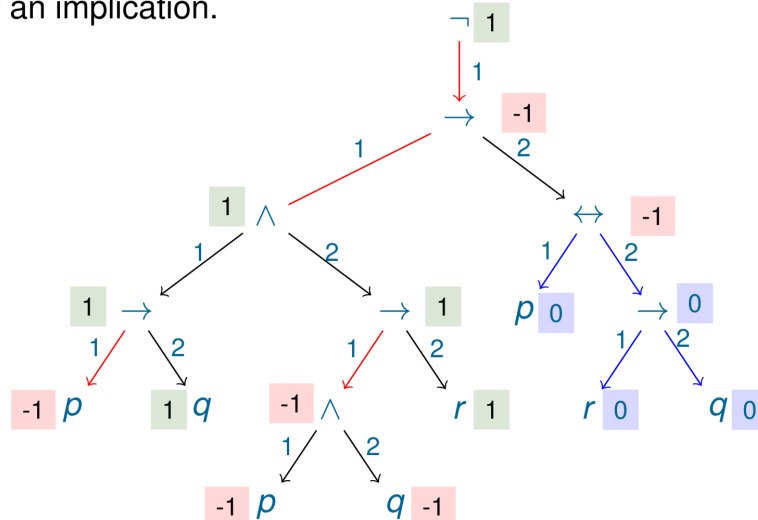
Visuelle Methode, um Polaritäten in einem Syntaxbaum zu bestimmen:

1. Färbe alle Kanten unterhalb einer Äquivalenz ($\leftrightarrow$) **blau**.
2. Färbe alle (noch ungefärbten) Kanten, die von einem $\neg$ oder der linken Seite eines $\rightarrow$ nach unten gehen, **rot**.
3. Auswertung für einen Knoten:
 - Mindestens eine blaue Kante darüber $\implies$ Polarität **0**.
 - Ungerade Anzahl roter Kanten darüber $\implies$ Polarität **-1**.
 - Gerade Anzahl roter Kanten (oder gar keine) darüber $\implies$ Polarität **1**.

The Coloring Algorithm for Determining Polarity

Another example: $\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \leftrightarrow (r \rightarrow q)))$

- Color in **blue** all arcs below an equivalence.
- Color in **red** all uncolored arcs going down from a negation or left-hand side of an implication.



- If a position has **at least one blue arc** above it, its polarity is **0**;
- **Otherwise**, its polarity is **-1** if it has an **odd number of red arcs** above it; its polarity is **1** if it has an **even number of red arcs** above it.

Monotonic Replacement & Pure Atoms

Monotonic Replacement (Monotones Ersetzen)

Sei eine Interpretation I , in der $I \models B_1 \rightarrow B_2$ gilt.

- Wenn Position π positive Polarität hat ($pol = 1$): $I \models A[B_1]_\pi \rightarrow A[B_2]_\pi$.
- Wenn Position π negative Polarität hat ($pol = -1$): $I \models A[B_2]_\pi \rightarrow A[B_1]_\pi$ (Richtung dreht sich um!).

Pure Atoms (Reine Atome)

Ein Atom p ist "pure" (rein) in einer Formel A , wenn alle Vorkommen von p in A **nur positiv** oder **nur negativ** sind.

Theorem für reine Atome (Pure Atom Theorem)

Das Finden reiner Atome vereinfacht die Erfüllbarkeitsprüfung extrem:

- Wenn p **nur positiv** Vorkommen hat: Formel A ist erfüllbar $\iff$ Formel ist erfüllbar, wenn man p durch $\top$ (Wahr) ersetzt ($A_p^\top$).
- Wenn p **nur negativ** Vorkommen hat: Formel A ist erfüllbar $\iff$ Formel ist erfüllbar, wenn man p durch $\perp$ (Falsch) ersetzt ($A_p^\perp$).

(Zusatz: Dies erlaubt es, Atome in der Formel deterministisch auszuwerten, ohne einen verzweigenden Baum aufbauen zu müssen. Ersetze das reine Atom, vereinfache die Formel, prüfe weiter).

Splitting Algorithmus mit Pure Atom Optimization:

$$G := \text{simplify}_{\text{with pure atoms}} \cdot (G)$$

Konjunktive Normalform (CNF) & DPLL

Um Algorithmen wie DPLL zu verwenden, muss die Formel in einer speziellen Struktur vorliegen.

Literale und Klauseln

- **Literal:** Ein Atom p (positives Literal) oder dessen Negation $\neg p$ (negatives Literal).
- **Komplementäres Literal:** Das Gegenteil eines Literals (bezeichnet als $\bar{L}$). Zu p ist es $\neg p$, zu $\neg p$ ist es p .
- **Klausel:** Eine Disjunktion ($\vee$) von Literalen: $L_1 \vee \dots \vee L_n$.

- *Empty clause (leere Klausel)*: $n = 0$, ist immer falsch ($\perp$).
- *Unit clause (Einheitsklausel)*: $n = 1$ (enthält nur genau ein Literal).
- *Horn clause (Horn-Klausel)*: Klausel mit höchstens einem positiven Literal (der Rest ist negiert).

Konjunktive Normalform (CNF)

Eine Formel ist in CNF, wenn sie ein "Und von Oders" ist, also eine Konjunktion von Klauseln.

$$A = \bigwedge_i \bigvee_j L_{i,j}.$$

- Eine Interpretation erfüllt eine CNF-Formel, wenn sie jede Klausel darin erfüllt.
- Eine Klausel ist erfüllt, wenn mindestens ein Literal darin wahr ist.

Komplexität von SAT-Problemen (k-SAT):

Prüfung, ob eine Menge von Klauseln der Länge k erfüllbar ist.

- **2-SAT**: In linearer Zeit entscheidbar.
- **3-SAT & allgemeines SAT**: NP-vollständig (Schwer lösbar).

SAT Roulette on CNF



We will generate random instances of **2-SAT** with **5-variables**.

You will bet on whether the resulting set of clauses is satisfiable or not.

- ▶ What will you bet on if we generate 5 clauses?
- ▶ What will you bet on if we generate 100 clauses?
- ▶ What will you bet on if we generate 15 clauses?

Um eine Formel in CNF zu wandeln, wendet man folgende Regeln an, bis keine mehr anwendbar ist:

1. Äquivalenzen auflösen: $A \leftrightarrow B \Rightarrow (\neg A \vee B) \wedge (\neg B \vee A)$.
2. Implikationen auflösen: $A \rightarrow B \Rightarrow \neg A \vee B$.
3. De Morgan (Negationen nach innen ziehen): $\neg(A \wedge B) \Rightarrow \neg A \vee \neg B$.
 - $\neg(A \vee B) \Rightarrow \neg A \wedge \neg B$.
4. Doppelte Negationen entfernen: $\neg\neg A \Rightarrow A$.
5. Distributivgesetz anwenden (Ausmultiplizieren):
 - Ziehe $\vee$ in $\wedge$ hinein: $(A_1 \wedge \dots \wedge A_m) \vee B \Rightarrow (A_1 \vee B) \wedge \dots \wedge (A_m \vee B)$.

Wann ist man fertig?

Die Formel enthält kein $\leftrightarrow$ und kein $\rightarrow$ mehr. Das $\neg$ steht nur noch direkt vor Atomen. Und das $\wedge$ steht nirgends mehr innerhalb des Wirkungsbereichs eines $\vee$.

Sinn der CNF-Transformation

Sie reduziert das komplexe Erfüllbarkeitsproblem (Satisfiability) beliebiger, beliebig tief verschachtelter Formeln auf das simple Problem, eine bloße Menge von einfachen Klauseln auszuwerten.

L6

Decision Procedures for Propositional Satisfiability

Einführung

Es gibt verschiedene, fortschrittliche Entscheidungsverfahren für die Erfüllbarkeit (Satisfiability) von aussagenlogischen Formeln:

- **Splitting** mit Polaritäts-Optimierungen
- **DPLL** (Davis-Putnam-Logemann-Loveland, 1962)
- **Resolution**
- **Semantic Tableaux**

- DPLL arbeitet auf einer eingeschränkten Menge von Formeln, weshalb wir zunächst Normalformen (Normal Forms) und deren Konvertierung betrachten müssen.

Konjunktive Normalform (CNF) & k-SAT

Satisfiability on CNF

Eine Interpretation I erfüllt eine Formel in CNF genau dann, wenn sie **jede einzelne Klausel** darin erfüllt.

Formelstruktur: $A = \bigwedge_i \bigvee_j L_{i,j}$

- Eine Interpretation erfüllt eine einzelne Klausel $L_1 \vee \dots \vee L_k$ genau dann, wenn sie **mindestens ein Literal** L_m in dieser Klausel erfüllt.
- **k-SAT**: Das Problem der Erfüllbarkeitsprüfung für Mengen von Klauseln $\{C_1, \dots, C_m\}$, wobei jede Klausel exakt k Literale enthält.

Komplexität

- **2-SAT**: Entscheidbar in linearer Zeit (linear time).
- **3-SAT**: NP-vollständig (NP-complete).
- **SAT (allgemein)**: NP-vollständig.

SAT Roulette on CNF



We will generate random instances of 2-SAT with 5-variables.

You will bet on whether the resulting set of clauses is satisfiable or not.

- ▶ What will you bet on if we generate 5 clauses?
- ▶ What will you bet on if we generate 100 clauses?
- ▶ What will you bet on if we generate 15 clauses?

CNF Transformation (Wiederholung der Äquivalenzregeln)

Um eine Formel in CNF umzuwandeln, wendet man schrittweise folgende Äquivalenzen an:

1. $A \leftrightarrow B \Rightarrow (\neg A \vee B) \wedge (\neg B \vee A)$
2. $A \rightarrow B \Rightarrow \neg A \vee B$
3. De Morgan: $\neg(A \wedge B) \Rightarrow \neg A \vee \neg B$ und $\neg(A \vee B) \Rightarrow \neg A \wedge \neg B$
4. Doppelte Negation: $\neg\neg A \Rightarrow A$
5. Distributivgesetz:

$$(A_1 \wedge \dots \wedge A_m) \vee B_1 \vee \dots \vee B_n \Rightarrow (A_1 \vee B_1 \vee \dots \vee B_n) \wedge \dots \wedge (A_m \vee B_1 \vee \dots \vee B_n)$$

Eine Formel, auf die **keine** dieser Regeln mehr anwendbar ist, ist in CNF. Das bedeutet, sie:

- enthält kein $\leftrightarrow$ und kein $\rightarrow$.
- enthält $\neg$ nur direkt vor Atomen (Literale).
- enthält kein $\wedge$ innerhalb des Wirkungsbereichs eines $\vee$.

≡ Beispiel einer vollständigen CNF-Transformation >

Startformel: $\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$

Nach Anwendung der Regeln erhält man die Klauselmenge:

$$(\neg p \vee q) \wedge (\neg p \vee \neg q \vee r) \wedge p \wedge \neg r$$

Die CNF-Transformation reduziert somit das Erfüllbarkeitsproblem von Formeln auf das von Klauselmengen.

Das Problem der Standard-CNF

↪ Exponentielles Wachstum (Exponential Blowup)

Es gibt Formeln, deren kürzeste CNF von **exponentieller Größe** ist.

Beispiel: $p_1 \leftrightarrow (p_2 \leftrightarrow (p_3 \leftrightarrow (p_4 \leftrightarrow (p_5 \leftrightarrow p_6))))$

Die Standard-Umwandlung in CNF würde hier extrem groß werden (z.B. 32 Kopien von p_6 erzeugen).

Ziel: Wir brauchen eine Normalform, die sich ohne exponentielles Wachstum (almost linear time) erstellen lässt.

Definitional Clause Form Transformation (Naming)

Die Idee: Wir vergeben "Namen" (neue Variablen) für nichttriviale Teilformeln.

- Man nimmt eine Teilformel A und führt eine neue aussagenlogische Variable n ein.
- Man fügt eine Definition hinzu: $n \leftrightarrow A$.
- Man ersetzt die Teilformel im Original durch n .
- **Wichtig:** Die neue Menge an Formeln hat dieselben Modelle (bezogen auf die ursprünglichen Variablen), ist aber **nicht logisch äquivalent** zur Originalformel (sie ist lediglich erfüllbarkeitsäquivalent / equisatisfiable).

Clausal Form

Eine Menge von Klauseln, die genau dann erfüllbar ist, wenn die Ursprungsformel A erfüllbar ist.

Algorithmus zur definitorischen Transformation

1. Wenn A die Form $C_1 \wedge \dots \wedge C_n$ hat (wobei alle C_i Klauseln sind), dann ist $S = \{C_1, \dots, C_n\}$.

2. Andernfalls: Führe für jede Teilformel B von A (die kein Literal ist) einen neuen Namen ein und verwende diesen.

Example

	subformula	definition	clauses
			n_1
n_1	$\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$	$n_1 \leftrightarrow \neg n_2$	$\neg n_1 \vee \neg n_2$ $n_1 \vee n_2$
n_2	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r)$	$n_2 \leftrightarrow (n_3 \rightarrow n_7)$	$\neg n_2 \vee \neg n_3 \vee n_7$ $n_3 \vee n_2$ $\neg n_7 \vee n_2$
n_3	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r)$	$n_3 \leftrightarrow (n_4 \wedge n_5)$	$\neg n_3 \vee n_4$ $\neg n_3 \vee n_5$ $\neg n_4 \vee \neg n_5 \vee n_3$
n_4	$p \rightarrow q$	$n_4 \leftrightarrow (p \rightarrow q)$	$\neg n_4 \vee \neg p \vee q$ $p \vee n_4$ $\neg q \vee n_4$
n_5	$p \wedge q \rightarrow r$	$n_5 \leftrightarrow (n_6 \rightarrow r)$	$\neg n_5 \vee \neg n_6 \vee r$ $n_6 \vee n_5$ $\neg r \vee n_5$
n_6	$p \wedge q$	$n_6 \leftrightarrow (p \wedge q)$	$\neg n_6 \vee p$ $\neg n_6 \vee q$ $\neg p \vee \neg q \vee n_6$
n_7	$p \rightarrow r$	$n_7 \leftrightarrow (p \rightarrow r)$	$\neg n_7 \vee \neg p \vee r$ $p \vee n_7$ $\neg r \vee n_7$

Convert the resulting formula (set of definitions) to CNF, based on the standard formula equivalence rules.

Optimised Definitional Clause Form Transformation (Polarity)

Man kann den Prozess weiter optimieren, indem man die **Polarität** (positives/negatives Vorkommen) von Teilformeln ausnutzt. Statt einer Äquivalenz ($\leftrightarrow$) reicht oft eine einfache Implikation ($\rightarrow$), was am Ende **weniger Klauseln** erzeugt.

Polaritäts-Regeln

- **Positive Polarität:** Wenn Teilformel B **nur positiv** in A vorkommt, verwende $n \rightarrow B$ (statt $n \leftrightarrow B$).
- **Negative Polarität:** Wenn B **nur negativ** vorkommt, verwende $B \rightarrow n$.

(Anmerkung: Dies ist logisch ausreichend, um die Erfüllbarkeit zu erhalten. Siehe Lemma Naming on Positive/Negative Polarity auf Slide 71/72)

Unit Propagation

Funktionsweise von Unit Propagation

Eine Methode, um Klauselmengen durch iteratives Anwenden deterministisch zu vereinfachen. Wenn eine Klauselmenge S eine **Unit Clause** (eine Klausel, die aus nur einem einzigen Literal L besteht) enthält, mache folgendes:

1. **Entferne** aus S jede Klausel der Form $L \vee C'$ (da L wahr ist, ist die ganze Klausel erfüllt).
2. **Ersetze** in S jede Klausel der Form $\bar{L} \vee C'$ durch die verkürzte Klausel C' (da $\bar{L}$ falsch ist, muss C' wahr sein, um die Klausel zu erfüllen).

Dies wiederholt man so lange, bis keine Unit Clauses mehr vorhanden sind. Oft lässt sich so die Unerfüllbarkeit (Unsatisfiability) feststellen, ohne raten zu müssen.

DPLL = Splitting + Unit Propagation

Der DPLL-Algorithmus kombiniert systematisches Raten (Splitting) mit Unit Propagation.

Der DPLL Algorithmus (Pseudocode)

```

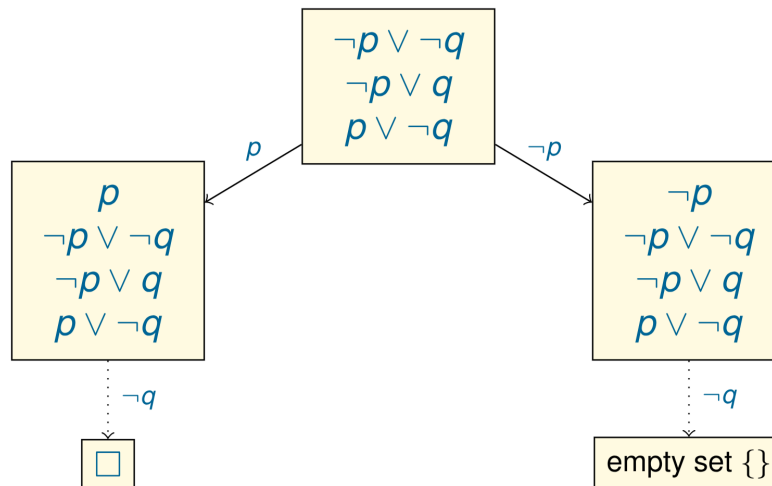
procedure DPLL(S)
  input: set of clauses S
  output: satisfiable or unsatisfiable
  parameters: function select literal
begin
  S := propagate(S) // Wende Unit Propagation an
  if S is empty then return satisfiable
  if S contains [] (empty clause) then return unsatisfiable
  L := select literal(S) // Heuristik zur Literal-Auswahl
  if DPLL(S u {L}) = satisfiable // Splitting (zuerst L=wahr)
    then return satisfiable
  else return DPLL(S u {-L}) // Splitting (dann L=falsch)
end
  
```

DPLL-Ausführungen lassen sich gut durch **DPLL-Bäume** veranschaulichen.

Das Modell (die Belegung) kann abgelesen werden, indem man alle ausgewählten Literale (Splitting) und die durch Unit Propagation abgeleiteten Literale auf dem Ast sammelt, der zur leeren

Klauselmenge führt.

DPLL. Example 2



The set of clauses is **satisfiable**.

Zwei Optimierungen für DPLL

1. **Tautologien entfernen:** Jede Klausel der Form $p \vee \neg p \vee C$ ist immer wahr (Tautologie) und kann gelöscht werden.
2. **Pure Literals entfernen:** Ein Literal L in S ist "pure" (rein), wenn S keine Klauseln der Form $\overline{L} \vee C$ (also das negierte Literal) enthält. Alle Klauseln mit einem puren Literal können einfach entfernt werden (indem man dem Literal den entsprechenden Wahrheitswert zuweist).

Running a SAT Solver (MiniSat)

- Ein simpler, aber effizienter SAT-Solver ist **MiniSat**.
- Input Format: **DIMACS**.

DIMACS Format Aufbau:

Plaintext

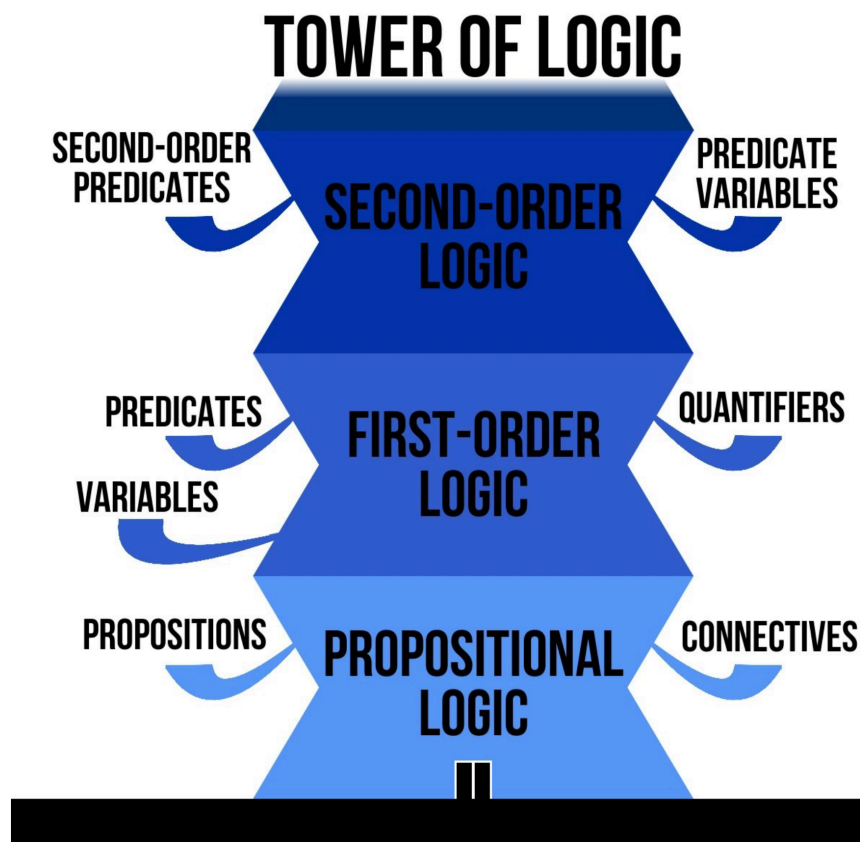
```
p cnf 3 4      // 3 Variablen, 4 Klauseln
1 0            // p1
-1 2 0         //  $\neg p_1 \vee p_2$ 
-1 -2 3 0      //  $\neg p_1 \vee \neg p_2 \vee p_3$ 
-2 -3 0        //  $\neg p_2 \vee \neg p_3$ 
```

(Zahlen stehen für die Variablen $p_1, p_2, \dots$, negative Zahlen bedeuten Negation. Jede Klausel wird mit einer 0 beendet)

L7

Tower of Logic

Recap



- **Propositional Logic (Aussagenlogik):** Basiert auf Propositionen und logischen Verknüpfungen (Connectives).
- **First-Order Logic (Prädikatenlogik 1. Stufe):** Erweitert um Prädikate, Variablen und Quantoren.
- **Second-Order Logic (Prädikatenlogik 2. Stufe):** Erweitert um Prädikate zweiter Stufe und Prädikatenvariablen.

Recap: Propositional Logic & Correctness

- **Korrektheit des Schließens:** Ein logischer Schluss ist korrekt, wenn die Schlussfolgerung (Conclusion) immer wahr ist, sobald die Prämissen wahr sind. (*Unabhängig davon, ob die Prämissen in der Realität immer wahr sind*).
- **Semantische Konsequenz ($KB \models Q$):** Q ist eine semantische Konsequenz aus einer Menge an Formeln Γ , wenn es **kein Gegenbeispiel** gibt.
 - Das bedeutet: Es gibt keine Interpretation I , die alle Prämissen wahr macht ($I(P_i) = 1$) und gleichzeitig die Schlussfolgerung falsch ($I(Q) = 0$).

- **Entscheidungsverfahren (Decision procedures):** Bisherige Ansätze waren semantisch (Wahrheitstabellen), Splitting mit Polaritätsoptimierungen und der DPLL-Algorithmus.

Proof Calculus (Beweiskalkül)

Was ist ein Kalkül?

Ein Kalkül besteht aus **Inferenzregeln** (Schlussregeln), die es ermöglichen, mechanisch und Schritt-für-Schritt neue Formeln abzuleiten.

- **Wichtig:** Dies geschieht **nur** anhand der Syntax (der Zeichenfolge der Formeln), nicht durch Semantik (Wahrheitswerte)!

- **Schreibweise:** $KB \vdash G$ (oder $F_1, \dots, F_n \vdash G$). Das bedeutet: G ist aus KB syntaktisch ableitbar.

Warum brauchen wir einen Kalkül?

- **Effizienz:** Es ist meist viel effizienter als das Aufstellen riesiger Wahrheitstabellen.
- **Beweisbarkeit statt nur Wahrheit:** Wir suchen nach einer formellen Begründung (Beweis), warum ein Schluss korrekt ist.
- **Erweiterbarkeit:** Kalküle lassen sich auf Prädikatenlogik (First-Order Logic) und nicht-klassische Logiken erweitern (Wahrheitstabellen funktionieren bei unendlichen Domänen nicht mehr).

Syntax vs. Semantik (Desiderata für einen Kalkül)

- **Semantik:** Interpretationen definieren wahre Formeln $\rightarrow$ **Entailment** ($KB \models A$).
- **Syntax:** Inferenzregeln leiten Formeln ab $\rightarrow$ **Derivation/Ableitung** ($KB \vdash A$).

Soundness & Completeness (Korrektheit & Vollständigkeit)

- **Soundness (Korrektheit):** $KB \vdash A \implies KB \models A$ (*Alles was abgeleitet wird, ist auch wirklich wahr "Nothing but truth"*).
- **Completeness (Vollständigkeit):** $KB \models A \implies KB \vdash A$ (*Alles was wahr ist, kann auch abgeleitet werden*).

Inferenzregeln und Resolution

Modus Ponens

- Regel:

$$\frac{A \rightarrow B \quad A}{B}$$

- Operiert direkt auf der Syntax.
- *Problem:* Ist zwar **sound**, aber **nicht complete** (nicht vollständig) für allgemeine Klauseln.

Resolution (Robinson 1965)

Grundlagen der Resolution

Die Resolution bildet die Basis für die Programmiersprache PROLOG. Sie wird auf Formeln in Klauselform (CNF - Konjunktive Normalform) angewendet.

- Propositional Resolution Rule:

$$\frac{C \vee p \quad \neg p \vee D}{C \vee D}$$

- Erzeugt eine neue Klausel (Resolvent: $C \vee D$), indem zwei Klauseln aufgelöst werden, die ein komplementäres Literal (p und $\neg p$) enthalten.
- Factoring Rule (Faktorisierung):

$$\frac{C \vee l \vee l}{C \vee l}$$

- *Intuitiv:* Entfernt redundante Literale aus einer Klausel.

Deduction vs. Refutation

Resolution ist **nicht vollständig für direkte Ableitungen (Derivations)**! (Man kann z.B. $p \vee \neg p$ nicht direkt aus einer leeren Wissensbasis resoliert ableiten).

Lösung: Wir verwenden Beweise durch Widerspruch (**Refutation**).

Refutation (Beweis durch Widerspruch)

- Theorem (Duality): $\Gamma \models Q \iff \Gamma \cup \{\neg Q\}$ ist **unerfüllbar (unsatisfiable)**.
- General Idea:
 1. Gegeben ist eine Wissensbasis $KB = \{Q_1, \dots, Q_n\}$ und eine zu beweisende Formel A .
 2. Wir negieren A und transformieren alles in CNF: $(Q_1 \wedge \dots \wedge Q_n \wedge \neg A)^{cnf}$.
 3. Wir nehmen an, diese Menge ist erfüllbar und suchen nach einem Widerspruch durch Resolution.
 4. Finden wir einen Widerspruch (die **leere Klausel** $\square$), war die Annahme falsch $\implies KB \models A$.

- **Vorteil:** Finden wir keinen Widerspruch (Algorithmus terminiert ohne $\square$), haben wir Erfüllbarkeit bewiesen (also $KB \models A$).

✓ Eigenschaften der Resolution

Der Resolutionskalkül (mit Factoring) ist **sound** und **refutational complete** (widerlegungsvollständig).

- **Soundness:** Wenn es eine Refutation (Widerspruch) gibt, dann ist KB unerfüllbar.
- **Completeness:** Wenn KB unerfüllbar ist, dann existiert garantiert eine Refutation (wir finden die leere Klausel $\square$).

Semantic Tableaux (Semantische Tableaux)

❗ Was sind Semantic Tableaux?

Eine Beweistechnik, die Wahrheitstabellen direkt widerspiegelt bzw. ausnutzt.

- Keine Transformation in Klauselform (CNF) nötig!
- Geht schrittweise vor und zerlegt die zu widerlegende Formel baumartig (tree-like).
- Funktioniert ebenfalls durch **Refutation** (Widerlegung).

Die Idee (Algorithmus)

1. Wir wollen prüfen, ob $P \models Q$.
2. Annahme (Widerspruch): Es gibt eine Interpretation, die Prämissen (P) **wahr (t)** und Schlussfolgerung (Q) **falsch (f)** macht.
3. Wir labeln die Formeln: $t : P$ und $f : Q$.
4. Wir wenden Inferenzregeln an, um die Formeln zu vereinfachen.
5. **Baumstruktur:**
 - Zeilen **untereinander** gehören zur selben Interpretation.
 - **Branching (Verzweigung):** Manche Regeln erzeugen parallele Äste. Verschiedene Äste repräsentieren verschiedene Interpretationen.
6. **Abschluss (Closed/Open):**
 - Ein Ast wird **geschlossen (closed)**, wenn er einen Widerspruch enthält (z.B. $t : A$ und $f : A$ gleichzeitig).
 - Ein Tableau ist **geschlossen (closed)** (Beweis erfolgreich!), wenn **alle** Äste geschlossen sind.
 - Ist das Tableau fertig entwickelt und es gibt noch **offene (open)** Äste, dann existiert ein Gegenbeispiel ($P \not\models Q$).

Tableau Regeln (Auszug)

Die Regeln spiegeln exakt die Logik der Wahrheitstabellen wider.

☰ Wichtige Tableau Inferenzregeln >

UND ($\wedge$)

- $t : A \wedge B \implies$ Füge $t : A$ und $t : B$ im selben Ast hinzu.
- $f : A \wedge B \implies$ Verzweige in $f : A \mid f : B$ (da nur eines falsch sein muss).

ODER ($\vee$)

- $t : A \vee B \implies$ Verzweige in $t : A \mid t : B$.
- $f : A \vee B \implies$ Füge $f : A$ und $f : B$ im selben Ast hinzu.

IMPLIKATION ($\rightarrow$)

- $t : A \rightarrow B \implies$ Verzweige in $f : A \mid t : B$. (Entspricht $\neg A \vee B$)
- $f : A \rightarrow B \implies$ Füge $t : A$ und $f : B$ im selben Ast hinzu.

NEGATION ($\neg$)

- $t : \neg A \implies f : A$
- $f : \neg A \implies t : A$

✓ Eigenschaften

- Jeder Beweisversuch terminiert (Aussagenlogik ist entscheidbar).
- Das Tableau-Kalkül ist sound und complete.

L8

Logik und Reasoning: Effizientes SAT-Solving & Horn-Klauseln

 Motivation

Die Erfüllbarkeit (Satisfiability) einer allgemeingültigen CNF-Formel ist generell **NP-vollständig**. Für bestimmte, eingeschränkte Formel-Arten lässt sich das Problem jedoch deutlich effizienter (in Polynomialzeit) lösen.

Horn-Klauseln

 Definition: Horn-Klausel

Eine Klausel ist genau dann eine **Horn-Klausel**, wenn sie **maximal ein positives Literal** enthält.

Beispiele für Horn-Klauseln:

- p_1 (Unit-Klausel, 1 positives Literal)
- $\neg p_1 \vee p_2$ (1 positives, 1 negatives Literal)
- $\neg p_1 \vee \neg p_2 \vee p_3$ (1 positives, 2 negative Literale)
- $\neg p_3 \vee \neg p_4$ (0 positive, 2 negative Literale)

Beispiele für NON-Horn-Klauseln (mehr als ein positives Literal):

- $p_1 \vee p_2$
- $p_1 \vee \neg p_2 \vee p_3$

Wichtige Eigenschaften von Horn-Klauseln

Sei F eine Formel in CNF, die nur aus Horn-Klauseln besteht:

1. Wenn **jede** Klausel in F mindestens ein negatives Literal enthält, ist die Formel immer **erfüllbar (SAT)**.
 - (Erklärung: Man kann einfach alle Variablen auf **0** bzw. **false** setzen, wodurch alle negierten Literale **true** werden und somit jede Klausel erfüllt ist.)
2. Wenn eine Klausel $C \in F$ **kein** negatives Literal enthält, dann ist sie zwangsläufig entweder:
 - eine **Unit-Klausel** (besteht nur aus einem einzigen positiven Literal)
 - die **leere Klausel** (was sofort zu einem Widerspruch bzw. UNSAT führt)

Algorithmus zur Lösung von Horn-Klauseln

🔗 Unit Propagation

Das Entfernen eines Literals aus einer Horn-Klausel ergibt immer wieder eine Horn-Klausel. Deshalb kann die Erfüllbarkeit einer Menge von Horn-Klauseln C vollständig und effizient durch **Unit Propagation** (Einheitsresolution) entschieden werden.

⚙️ Algorithmus: **HornSolve(F)**

Solange die Formel F nicht leer ist:

1. Wähle eine Unit-Klausel C aus F .
2. Wenn keine solche Klausel existiert $\rightarrow$ **return SAT**.
3. Sei v das Literal in C (es muss positiv sein, damit es wahr wird: $v \leftarrow C[0]$).
4. Entferne alle Klauseln aus F , die v enthalten (*da diese Klauseln nun durch v erfüllt sind*).
5. Entferne das Literal $\neg v$ aus allen verbleibenden Klauseln (*da $\neg v$ durch die Zuweisung falsch wird*).
6. Tritt dabei eine leere Klausel auf $\rightarrow$ **return UNSAT**.

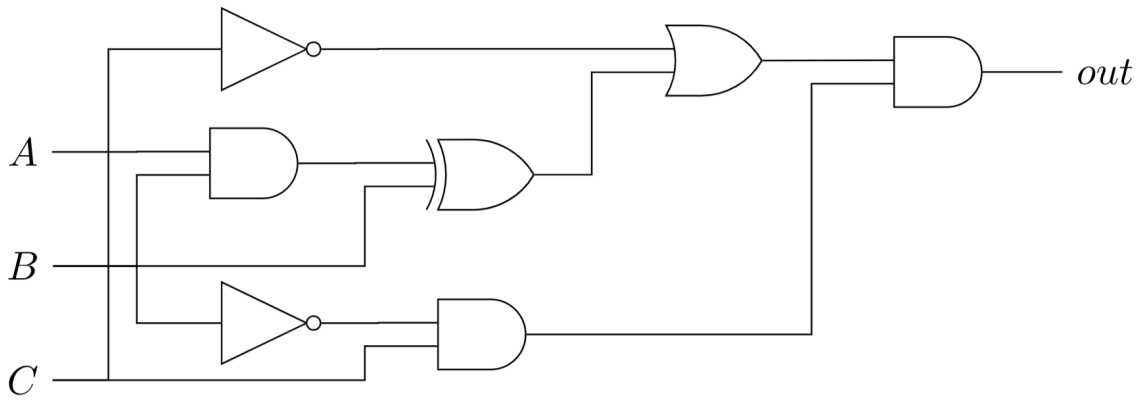
Was passiert am Ende der Unit Propagation?

Wendet man Unit Propagation auf eine Menge von Horn-Klauseln C an, erhält man eine neue (vereinfachte) Menge von Horn-Klauseln C' . Dabei gibt es zwei Fälle:

- **Fall 1:** C' enthält die **leere Klausel** $\rightarrow C'$ (und damit die Ursprungsformel C) ist **unsatisfiable (UNSAT)**.
- **Fall 2:** C' enthält keine leere Klausel und auch keine Unit-Klauseln mehr.
 - Dann hat jede verbleibende Klausel in C' mindestens zwei Literale.
 - Da es Horn-Klauseln sind, muss jede dieser Klauseln zwingend mindestens ein **negatives Literal** enthalten.
 - Setzt man nun alle noch unbelegten Variablen auf **0**, ist C' automatisch erfüllt.

Zusammenfassendes Beispiel: Dead Circuit (Toter Stromkreis)

Ziel ist es, herauszufinden, ob der Output eines bestimmten Schaltkreises immer **0** ist (was bedeuten würde, dass das letzte Logikgatter "tot" ist und entfernt werden kann).



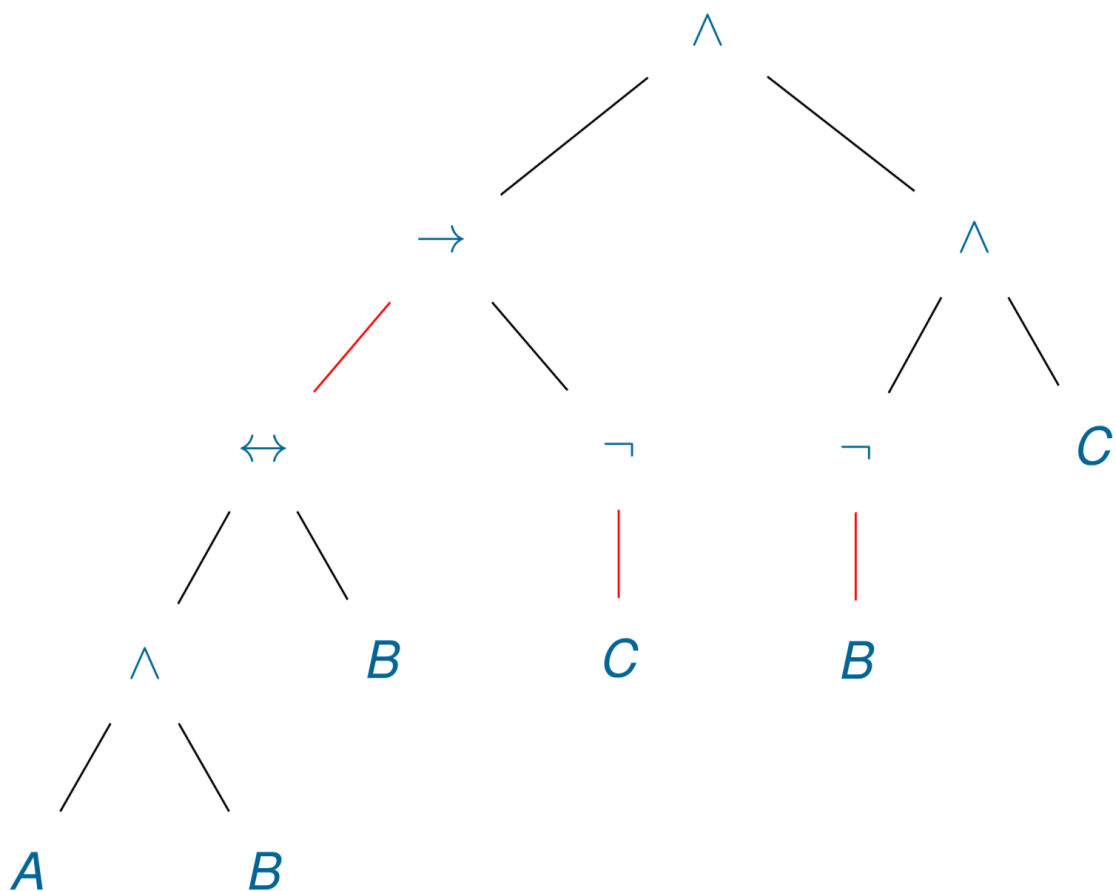
Die zugehörige aussagenlogische Formel für diesen Schaltkreis lautet:

$$(((A \wedge B) \leftrightarrow B) \rightarrow \neg C) \wedge (\neg B \wedge C)$$

Schritt 1: Parse Tree (Syntaxbaum) erstellen

Hier wird die Formel in ihre hierarchische Baumstruktur zerlegt.

$$(((A \wedge B) \leftrightarrow B) \rightarrow \neg C) \wedge (\neg B \wedge C)$$



Schritt 2: CNF-Transformation (Tseitin-Transformation)

Um die Formel in einen SAT-Solver füttern zu können, wird sie in Konjunktive Normalform (CNF) umgewandelt. Dabei erhält jeder Teilbaum eine eigene Hilfsvariable (n_0 bis n_4), um eine exponentielle Vergrößerung der Formel zu vermeiden.

	Subformula	Renamed	Clauses
			n_0
n_0	$((A \wedge B) \leftrightarrow B) \rightarrow \neg C) \wedge (\neg B \wedge C)$	$n_0 \rightarrow n_1 \wedge n_4$	$\neg n_0 \vee n_1$ $\neg n_0 \vee n_4$
n_1	$((A \wedge B) \leftrightarrow B) \rightarrow \neg C$	$n_1 \rightarrow n_2 \rightarrow \neg C$	$n_1 \wedge \neg C$ $\neg n_2 \wedge \neg C$
n_2	$(A \wedge B) \leftrightarrow B$	$n_2 \leftarrow n_3 \leftrightarrow B$	$n_2 \vee n_3 \vee B$ $n_2 \vee \neg n_3 \vee \neg B$
n_3	$A \wedge B$	$n_3 \leftrightarrow A \wedge B$	$\neg n_3 \vee A$ $\neg n_3 \vee B$ $n_3 \vee \neg A \vee \neg B$
n_4	$\neg B \wedge C$	$n_4 \rightarrow (\neg B \wedge C)$	$\neg n_4 \vee \neg B$ $\neg n_4 \vee C$

Schritt 3: DPLL Algorithmus anwenden

Durch iterative Anwendung von Unit-Propagation (Zuweisung von $n_0, n_1, n_4, \neg B, C, \neg n_3, n_2$) wird nach und nach die Formel evaluiert.

- Am Ende lässt sich ein Widerspruch ableiten.
- **Schlussfolgerung:** Die Formel ist UNSAT. Das bedeutet, das Output-Signal ist immer 0 und das Kabel kann sicher aus dem Design entfernt werden.

Praxis: SAT-Solver ausführen



Ein sehr simpler, aber in der Praxis sehr effizienter und bekannter SAT-Solver ist [MiniSat](#).

Das DIMACS Eingabeformat

Damit ein SAT-Solver eine Formel verarbeiten kann, wird standardmäßig das [DIMACS-Format](#) verwendet.

Very simple but efficient SAT solver: MiniSat,
<http://minisat.se/>.

$$\begin{array}{c}
 p_1 \\
 \neg p_1 \vee p_2 \\
 \neg p_1 \vee \neg p_2 \vee p_3 \\
 \neg p_2 \vee \neg p_3
 \end{array}$$

DIMACS input format:

```

p cnf 3 4
1 0
-1 2 0
-1 -2 3 0
-2 -3 0

```

Aufbau des DIMACS-Formats:

- Die erste Zeile definiert das Format und die Problemgröße: `p cnf <Anzahl_Variablen> <Anzahl_Klauseln>` (z.B. `p cnf 3 4`).
- Variablen werden als **fortlaufende positive Ganzzahlen** dargestellt (1, 2, 3, ...).
- Negationen von Variablen werden durch ein **Minuszeichen** davor markiert (-1, -2, ...).
- Jede Klausel steht logisch in einer eigenen Zeile (bzw. ist getrennt) und **muss zwingend mit einer 0 am Ende abgeschlossen werden**.

≡ Anwendungsbeispiel: Sudoku als SAT-Problem

SAT-Solver lassen sich wunderbar verwenden, um Logikrätsel wie Sudoku extrem schnell zu lösen.

Dazu müssen lediglich die Spielregeln in aussagenlogische Klauseln (CNF) übersetzt werden:

- In jedes leere Feld muss eine Ziffer von 1 bis 9.
- Jede Reihe muss jede Ziffer genau einmal enthalten.
- Jede Spalte muss jede Ziffer genau einmal enthalten.
- Jedes 3x3 Quadrat muss jede Ziffer genau einmal enthalten.

4	1	8	7	9	5	6	3	2
6	3	9	8	2	1	5	7	4
5	2	7	3	6	4	1	8	9
9	5	2	4	3	6	8	1	7
1	4	6	5	8	7	9	2	3
7	8	3	2	1	9	4	5	6
2	6	1	9	5	3	7	4	8
3	7	5	6	4	8	2	9	1
8	9	4	1	7	2	3	6	5

L9

First-Order Logic (FOL) - Prädikatenlogik

1. Motivation: Von Aussagenlogik zu FOL

- In der reinen Aussagenlogik hat jede atomare Proposition (z.B. *SocratesIsHuman*) einen unabhängigen Wahrheitswert.
 - **Das Problem:** Klassische generelle Aussagen wie "Alle Menschen sind sterblich" können nicht generalisiert abgebildet werden, sondern nur für spezifische Individuen miteinander in Verbindung gebracht werden.
 - **Die Lösung (FOL):** Die Prädikatenlogik führt **Terme** ein, um auf Objekte zu verweisen, und **Prädikate**, um Eigenschaften dieser Objekte auszudrücken.
 - Über **Quantoren** ($\forall, \exists$) lassen sich Wahrheitswerte von Prädikaten für *alle* oder *einige* Objekte erzwingen.
-

2. Syntax der Prädikatenlogik

Signatur (Das Vokabular)

Um Formeln zu bilden, muss zuerst eine Signatur festgelegt werden. Diese besteht aus drei unendlichen, paarweise disjunkten Alphabeten:

- $\mathcal{V}$ (**Variablen**): Platzhalter für Objekte (z.B. x, y). (*Achtung: Dies sind keine aussagenlogischen Variablen!*)
- $\mathcal{F}$ (**Funktionssymbole**): Erlauben es, Objekte implizit zu referenzieren (z.B. $fatherOf(socrates)$).
 - Jedes $f \in \mathcal{F}$ hat eine feste Stelligkeit (Arity) $n \geq 0$.
 - **Konstanten** sind Funktionssymbole mit Stelligkeit 0 (sie repräsentieren ein einzelnes, spezifisches Objekt, z.B. *socrates*).
- $\mathcal{P}$ (**Prädikatssymbole**): Drücken Eigenschaften oder Relationen aus.
 - Jedes $P \in \mathcal{P}$ hat eine feste Stelligkeit $n \geq 0$.

Terme

Terme referenzieren Objekte im "Universe of Discourse" (der Domäne). Sie werden induktiv definiert:

- Jede Variable $x \in \mathcal{V}$ ist ein Term.
- Jede Konstante ($f \in \mathcal{F}$ mit Arity 0) ist ein Term.
- Wenn $f \in \mathcal{F}$ die Stelligkeit $n > 0$ hat und $t_1, \dots, t_n$ Terme sind, dann ist auch $f(t_1, \dots, t_n)$ ein Term.

Formeln

Formeln kombinieren Terme und Prädikate zu logischen Aussagen.

- **Atome (Atomare Formeln):** Haben die Form $P(t_1, \dots, t_n)$, wobei P ein Prädikat der Stelligkeit n ist und $t_1, \dots, t_n$ Terme sind. (*Prädikate mit Stelligkeit 0 verhalten sich wie aussagenlogische Variablen.*)
- **Induktiver Aufbau von Formeln:**
 - Jedes Atom ist eine Formel.
 - Wenn ϕ, ψ Formeln sind, dann sind auch $\neg\phi, (\phi \wedge \psi), (\phi \vee \psi), (\phi \rightarrow \psi)$ Formeln.
 - Wenn x eine Variable und ϕ eine Formel ist, dann sind auch $(\forall x \phi)$ und $(\exists x \phi)$ Formeln.

Bindungsstärke (Präzedenz)

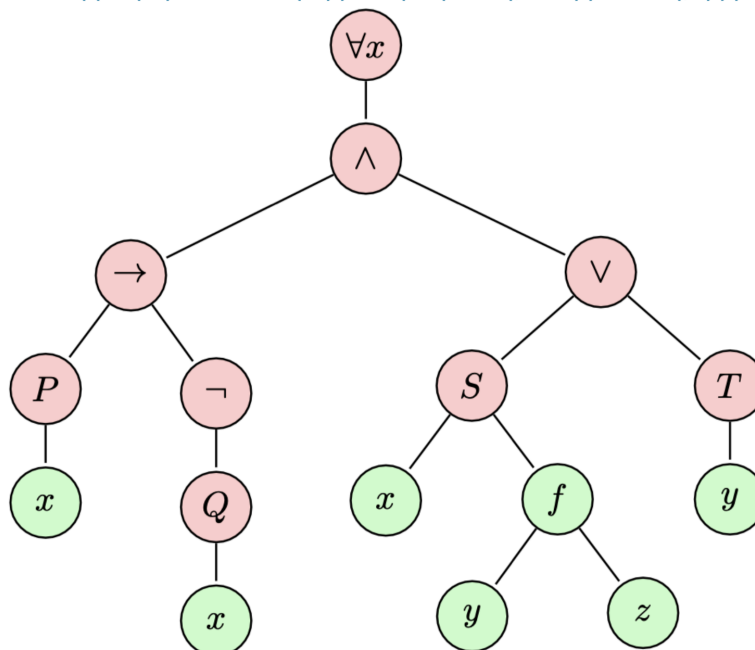
Um Klammern zu sparen, gilt folgende Priorität (von stark nach schwach bindend):

1. $\neg, \forall, \exists$ (stärkste Bindung)
2. $\wedge, \vee$
3. $\rightarrow$ (schwächste Bindung)

Scope, Variablen & Subformeln

Formula trees and Subformulas

$$\forall x((P(x) \rightarrow \neg Q(x)) \wedge (S(x, f(y, z)) \vee T(y)))$$



An occurrence of x is **bound** iff it has an upward path to $\forall x$ or $\exists x$.

- **Subformeln:** Die rekursiven Bestandteile einer Formel (z.B. sind ϕ und ψ Subformeln von $\phi \wedge \psi$).
 - **Quantifier Scope (Geltungsbereich):** In $\forall x \phi$ und $\exists x \phi$ ist ϕ der Scope des Quantors für x .
 - **Gebundene vs. Freie Variablen:**
 - Ein Vorkommen von x ist **gebunden**, wenn es innerhalb des Scopes eines Quantors ($\forall x$ oder $\exists x$) liegt (im Syntaxbaum einen Pfad nach oben zum Quantor hat).
 - Ein Vorkommen ist **frei**, wenn es durch keinen Quantor gebunden wird.
 - **Sentence (Satz / Geschlossene Formel):** Eine Formel, die **keine** freien Variablen enthält.
-

3. Semantik der Prädikatenlogik

Was bedeutet Semantik hier?

In der Aussagenlogik reichte es, Variablen wahr/falsch zuzuweisen. In der FOL müssen wir Objekten (Termen) eine Bedeutung geben und festlegen, welche Objekt-Kombinationen ein Prädikat erfüllen.

Strukturen $\mathcal{M}$ (Die "Mögliche Welt")

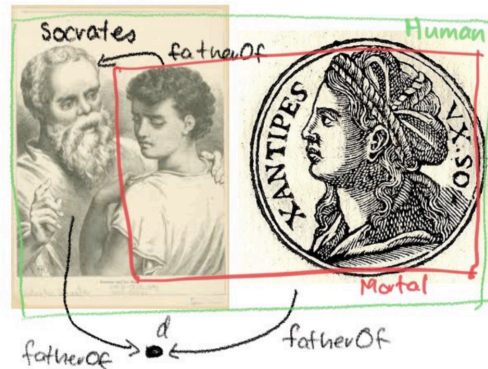
Eine Struktur beschreibt den Zustand einer Welt. Sie ist ein Paar $\mathcal{M} = (D, I)$:

- **D (Domain / Universe of Discourse):** Eine nicht-leere Menge von Objekten, über die wir sprechen.
- **I (Interpretation):** Eine Funktion, die dem Vokabular Bedeutung zuweist:
 - **Konstanten c :** Werden auf ein konkretes Objekt $I(c) \in D$ gemappt (Schreibweise: $c^{\mathcal{M}}$).
 - **Funktionen f (Stelligkeit n):** Werden auf **totale Funktionen** $I(f) : D^n \rightarrow D$ gemappt. *(Jede Funktion muss für jeden Input aus D exakt ein Output-Objekt in D definieren.)*
 - **Prädikate P (Stelligkeit n):** Werden auf eine Relation $I(P) \subseteq D^n$ gemappt (eine Menge von n -Tupeln aus D).

Meaning of predicates

Vocabulary: constant unary function unary predicates
 socrates *fatherOf* *Human*, *Mortal*

Predicates correspond to properties of the objects



The structure determines which atomic formulas are true.

- ▶ E.g., *Human(socrates)* is true
- ▶ *Mortal(socrates)* and *Mortal(fatherOf(socrates))* are false

7/24

≡ Herbrand-Struktur

Ein Spezialfall der Tarski-Semantik. Hier ist die Domäne D einfach die Menge aller möglichen **Grundterme** (Terme ohne Variablen). Jeder Term wird einfach auf "sich selbst" (als Objekt) abgebildet ($t^M = t$).

Variablenbelegung (Variable Assignment) l

Um Formeln mit *freien Variablen* auszuwerten, brauchen wir eine "Look-up Table".

- Eine Variablenbelegung $l : \mathcal{V} \rightarrow D$ weist jeder Variablen ein Objekt aus der Domäne zu.
- **Notation** $l[d/x]$: Entspricht der Belegung l , mit der einzigen Ausnahme, dass der Variablen x das spezifische Objekt d zugewiesen wird.

Interpretationen & Erfüllbarkeit (Satisfaction)

Eine **Interpretation** ist ein Paar $(\mathcal{M}, l)$.

Die Erfüllbarkeit einer Formel wird induktiv bestimmt:

- **Atome**: $(\mathcal{M}, l) \models P(t_1, \dots, t_n)$ gilt, wenn das Tupel der ausgewerteten Terme in der Menge P^M liegt.

- **Logische Verknüpfungen** ($\neg, \wedge, \vee, \rightarrow$): Werden klassisch wie in der Aussagenlogik ausgewertet (z.B. $(\mathcal{M}, l) \models \phi \wedge \psi$ gdw. ϕ und ψ erfüllt sind).
- **Allquantor**: $(\mathcal{M}, l) \models \forall x \phi$ gilt genau dann, wenn für *alle* $d \in D$ die Formel ϕ unter der modifizierten Belegung $l[d/x]$ wahr ist.
- **Existenzquantor**: $(\mathcal{M}, l) \models \exists x \phi$ gilt genau dann, wenn für *mindestens ein* $d \in D$ die Formel ϕ unter der Belegung $l[d/x]$ wahr ist.

✓ Modelle

Wenn eine Formel ϕ ein **Sentence** ist (also keine freien Variablen hat), ist die konkrete Belegung l komplett irrelevant. Gilt in diesem Fall $(\mathcal{M}, l) \models \phi$, so schreibt man einfach $\mathcal{M} \models \phi$ und nennt die Struktur $\mathcal{M}$ ein **Modell** für ϕ .

